

Übungsklausur 1 — Web-Engineering 1

Stil: Dozenten-Probeklausur (TINF21B2) · 75 Punkte · Bearbeitungszeit ~60 Min · MUSTERLÖSUNG

Aufgabe 1 (HTML)

25 Punkte

a) 4 P: Beschreiben Sie die Grundbausteine eines HTML-Dokuments und nennen Sie die dazugehörigen HTML-Tags.

Lösung: `<!DOCTYPE html>` (HTML5-Deklaration), Wurzelement `<html>`, Kopf `<head>` (mit `<title>`, `<meta charset>`) und Körper `<body>` mit dem sichtbaren Inhalt.

b) 4 P: Worin unterscheiden sich Block- und Inline-Elemente? Nennen Sie jeweils ein Beispiel-Tag.

Lösung: Block: nimmt die volle Zeilenbreite und beginnt eine neue Zeile (z.B. `<div>`, `<p>`). Inline: nur so breit wie der Inhalt, bleibt im Textfluss (z.B. `<span>`, `<a>`).

c) 1 P: Geben Sie ein Beispiel für eine korrekte Schachtelung von Tags an.

Lösung: `<p>Text <b>fett</b> weiter</p>` — das innere Tag wird zuerst geschlossen.

d) 4 P: Nennen Sie zwei globale Attribute, die jedem Tag zugewiesen werden können, und ihren Verwendungszweck.

Lösung: `id` (eindeutiger Bezeichner eines Elements), `class` (Klassenzuordnung für CSS/JS). Ebenfalls global: `style`, `title`.

e) 2 P: Geben Sie ein HTML-Fragment (ohne CSS) an, das eine Überschrift „Kurs“ und darunter einen Absatz mit einem fett ausgezeichneten Wort darstellt.

Lösung: `<h3>Kurs</h3><p>Dies ist <b>wichtig</b>.</p>`

f) 3 P: Geben Sie ein HTML-Fragment für eine ungeordnete Liste mit drei Einträgen an (ohne CSS).

Lösung: `<ul><li>Eins</li><li>Zwei</li><li>Drei</li></ul>`

g) 2 P: Benennen Sie einen Bestandteil einer HTTP-URL und machen Sie ein Beispiel.

Lösung: z.B. Protokoll (`https`) oder Host (`example.com`) oder Pfad (`/kurse/it`).

h) 5 P: Benennen Sie zwei mögliche Tags, die im HTML-`<head>` verwendet werden dürfen, und beschreiben Sie deren Funktion.

Lösung: `<title>` = Titel im Browser-Tab/Suchergebnis (Pflicht). `<meta charset="UTF-8">` = Zeichenkodierung (Umlaute). Auch: `<link rel="stylesheet">`, `<script>`.

Aufgabe 2 (CSS)

18 Punkte

a) 6 P: Benennen Sie die drei Teile der Syntax einer CSS-Regel und beschreiben Sie, wofür sie verwendet werden.

Lösung: **Selektor** (wer wird verändert), **Eigenschaft/Property** (was), **Wert/Value** (wie). Form: `selektor { eigenschaft: wert; }`.

b) 2 P: Beschreiben Sie eine Variante, wie CSS-Regeln kombiniert werden können.

Lösung: Mehrere Selektoren durch Komma teilen sich eine Deklaration: `h1, h2 { color: red; }`.

c) 2 P: Beschreiben Sie eine Methode, CSS in ein HTML-Dokument einzubinden.

Lösung: Extern über `<link rel="stylesheet" href="stil.css">` im `<head>` (seitenübergreifend nutzbar).

d) 4 P: Geben Sie für das folgende HTML-Dokument den „top“- und „left“-Wert beider div-Elemente in Pixel an (der Browser fügt keine weiteren Abstände/Rahmen hinzu).

```
#blau { position:absolute; background:blue;
        top:15px; left:25px; width:100px; height:100px; }
#rot  { position:relative; background:red;
        top:10px; left:10px; width:100px; height:100px; }

<div id="blau"><div id="rot"></div></div>
```

Lösung: **#blau** (absolute): top **15px**, left **25px** (relativ zum Seitenrand). **#rot** (relative): Normalposition = Ursprung von #blau (25,15), verschoben um top+10/left+10 → top **25px**, left **35px**.

e) 4 P: Gegeben sei der folgende HTML-Body. Welche Elemente (IDs) werden mit den Selektoren angesprochen? Schreiben Sie die IDs hinter den Selektor.

```
<h1 id="t1">T</h1>
<p id="s1" class="rot">A</p>
<ol id="l1" class="rot">
  <li id="e1">1</li>
  <li id="e2"><b id="fett">2</b></li>
  <li id="e3">3</li>
</ol>
<p id="s2" class="rot blau">B</p>
```

Selektoren: #s1 · .rot · ol[class="rot"] · [class~="rot"]
· #e2 :only-child · :first-child · ol > li · li + li

Lösung: #s1 → s1 · .rot → s1, l1, s2 · ol[class="rot"] → l1 · [class~="rot"] → s1, l1, s2 · #e2 :only-child → fett · :first-child → t1, e1, fett · ol > li → e1, e2, e3 · li + li → e2, e3

Aufgabe 3 (JavaScript)

15 Punkte

a) 2 P: Wofür wird JavaScript auf Webseiten verwendet?

Lösung: Für clientseitige Interaktivität/Dynamik: DOM manipulieren, auf Events reagieren, Formulare validieren, Daten per AJAX/Fetch nachladen.

b) 2 P: Beschreiben Sie eine Funktion der Sandbox, in der JavaScript im Browser ausgeführt wird.

Lösung: Isolation vom System: JS darf u.a. nicht auf das lokale Dateisystem zugreifen, nicht auf Daten einer fremden Origin (Same-Origin-Policy) und keine OS-Befehle ausführen.

c) 1 P: Welchen Wert bekommen ungesetzte Funktionsparameter?

Lösung: undefined .

d) 5 P: Gegeben sei folgendes Codesegment. Welchen Wert haben x1–x5? Geben Sie Objekt-/Array-Werte in JSON-Notation an.

```
let a="Hallo"; let b=5;
let c = {"p":10,"q":20,"r":30};
function Rechner(d,e){
  let a=0;
  b = d.length+2;
  for(const k in e){ a = a+e[k]; }
  this.s=a; this.t=d+" Welt";
  return a;
}
let x1 = new Rechner(a,c);
let x2 = Rechner("Hi",c);
let x3 = a; let x4 = [c]; let x5 = b;
```

Lösung: x1 = {"s":60,"t":"Hallo Welt"} (new → Objekt, primitiver return ignoriert) · x2 = 60 (Plain-Call → Rückgabewert) · x3 = "Hallo" · x4 = [{"p":10,"q":20,"r":30}] · x5 = 4 (globales b zuletzt von Rechner("Hi",c) : "Hi".length+2 = 4).

e) 5 P: Geben Sie ein JavaScript-Fragment an, das einen Listeneintrag mit dem Inhalt „IT2024“ und dem id-Attribut „fach“ erzeugt und in eine bestehende Liste mit der id „kurse“ einfügt (Position egal).

Lösung: `const li=document.createElement("li"); li.id="fach"; li.textContent="IT2024"; document.getElementById("kurse").appendChild(li);`

Aufgabe 4 (WebServer)

4 Punkte

a) 2 P: Benennen Sie zwei Aufgaben, die ein WebServer übernimmt.

Lösung: Zwei von: HTTP-Requests entgegennehmen und beantworten; statische Ressourcen ausliefern bzw. an Programmcode delegieren (dynamische Erzeugung); Zugriffe protokollieren.

b) 2 P: In welchem Prozessmodell wird CGI verwendet? Beschreiben Sie die Funktionsweise.

Lösung: CGI läuft im Modell **getrennter Prozesse**: Pro Anfrage startet der Webserver einen eigenen, neuen Prozess, der die Ausgabe (HTML) erzeugt und zurückliefert — isoliert, aber mit Prozess-Start-Overhead.

Aufgabe 5 (NodeJS)

13 Punkte

a) 2 P: Beschreiben Sie zwei Vorteile des Model-2-Entwurfsmusters gegenüber dem Model-1-Entwurfsmuster.

Lösung: Zwei von: View und Controller sind in getrennten Dateien (Kapselung der Logik); je Komponente kann die passende Technologie verwendet werden; bessere Wartbarkeit/Testbarkeit.

b) 3 P: Schreiben Sie einen Request-Handler, der den Client auf „https://example.com/“ weiterleitet, sobald eine HTTP-GET-Anfrage an die URL „/weiterleitung“ empfangen wird. Die Variable „app“ (Express-Instanz) ist initialisiert.

Lösung: `app.get("/weiterleitung", (req, res) => res.redirect("https://example.com/"));`

c) 3 P: Welche drei Eigenschaften haben die meisten Ansätze zur Generierung von HTML gemeinsam?

Lösung: Die meisten Ansätze haben gemeinsam: (1) ein Template/HTML-Gerüst mit Platzhaltern, (2) Trennung von Layout und Daten, (3) beim Rendern werden die Platzhalter durch konkrete Daten ersetzt → fertiges HTML.

d) 4 P: Benennen Sie zwei Nummernkreise der möglichen HTTP-Antwort-Status und beschreiben Sie deren Einsatzzweck.

Lösung: z.B. **2xx** = Erfolg (200 OK) · **4xx** = Client-Fehler (404 Not Found). Weitere: 3xx Umleitung (301/302), 5xx Server-Fehler (500).